SELF-HEALING INFRASTRUCTURE

Automated NGINX Recovery using Prometheus, Alertmanager and Ansible

Technical Project Documentation | Local Ubuntu + VirtualBox Lab

1. Project Overview

This project demonstrates an automated monitoring and recovery workflow for an NGINX web server running in Docker on an Ubuntu virtual machine. Prometheus continuously evaluates service health through the Prometheus Blackbox Exporter. When the HTTP probe fails for the configured duration, Prometheus raises an alert and sends it to Alertmanager. Alertmanager forwards the firing alert to a Flask webhook, which invokes an Ansible playbook to restart the NGINX container and verify that it is running.

2. Objectives

- Monitor NGINX availability and HTTP response time.
- Detect service failure using Prometheus alert rules.
- Route alerts through Alertmanager to a webhook receiver.
- Trigger automated recovery with Ansible and Docker.
- Visualize service health and metrics in Grafana.
- Demonstrate an end-to-end incident detection and recovery process in a local lab.

3. Tools and Technologies

Ubuntu Linux and VirtualBox provide the lab environment. Docker Compose runs NGINX, Prometheus, Blackbox Exporter, Alertmanager and Grafana as containers. Prometheus collects metrics and evaluates alert rules; Blackbox Exporter probes the NGINX HTTP endpoint. Alertmanager routes notifications. A Python Flask application receives webhook requests and starts Ansible. Ansible executes the Docker restart and checks container state. Grafana displays monitoring panels using Prometheus as its data source.

4. Architecture and Workflow

1) Blackbox Exporter sends an HTTP GET probe to http://nginx:80 over the Docker network.
2) Prometheus scrapes the probe metrics every 15 seconds and evaluates the NginxDown rule (probe_success == 0, sustained for 15 seconds).
3) Prometheus sends the firing alert to Alertmanager on port 9093.
4) Alertmanager posts the alert payload to the Flask endpoint at /webhook on host port 5000.
5) Flask checks the alert name and firing status, then runs the Ansible recovery playbook.
6) Ansible restarts the nginx-server container and verifies that Docker reports it as running.
7) The next successful probe reports probe_success = 1; Grafana reflects the recovered service state.

IMPLEMENTATION, VALIDATION & CONCLUSION

5. Main Components and Ports

NGINX: host port 8085 → container port 80 (application under monitoring).
Prometheus: host port 9090 (metrics, queries and alert evaluation).
Blackbox Exporter: host port 9115 (HTTP probing).
Alertmanager: host port 9093 (alert routing and status).
Flask webhook: host port 5000 (receives /webhook requests).
Grafana: host port 3000 (dashboards and visualizations).
Ansible: local automation tool; restarts the Docker container named nginx-server.

6. Implementation and Run Instructions

From the project directory, start the monitoring stack with: sudo docker compose up -d. Start the webhook in a separate Ubuntu terminal with: python3 webhook/app.py. Keep that process running while testing. Open the NGINX page at http://localhost:8085, Prometheus at http://localhost:9090, Alertmanager at http://localhost:9093 and Grafana at http://localhost:3000. Configure Prometheus as Grafana's data source using the container-network URL http://prometheus:9090 when Grafana shares the Compose network. The dashboard can display probe_success, probe_duration_seconds and Prometheus health.

7. Testing and Expected Results

Healthy state: curl -I http://localhost:8085 should return HTTP 200; the Prometheus query probe_success should return 1; Grafana should show NGINX UP / HTTP OK. Failure simulation: run sudo docker stop nginx-server. After the probe and alert timing windows, Prometheus should show NginxDown as firing and Alertmanager should receive it. The Flask log should show recovery execution and a successful Ansible play recap. Docker should show nginx-server running again, and curl -I http://localhost:8085 should return HTTP 200. The observed lab run confirmed webhook delivery and successful Ansible restart.

8. Security, Limitations and Future Improvements

This is a learning lab, not a production-hardened deployment. Flask's built-in development server should be replaced with a production WSGI server. Protect the webhook with authentication and network restrictions. The current sudoers rule permitting passwordless ansible-playbook execution is broad; production should use a narrowly scoped root-owned wrapper and least-privilege permissions. Add a lock/cooldown to prevent overlapping or repeated recovery jobs, structured audit logs, persistent volumes, pinned image versions, health checks and automated tests. Add TLS and secret management where appropriate. Validate the exact sudoers, webhook and container-network configuration before deploying outside the local VM.

9. Conclusion

The project demonstrates a practical self-healing pattern: observe service health, detect an incident, notify an automation endpoint, execute a recovery action and verify the result. It combines monitoring, alerting, container operations and configuration automation in a reproducible local environment.